

GPU-PIM Workload Bottleneck Analyzer

A reproducible GPU, memory-system, and PIM/NMP architecture exploration project

Portfolio Technical Report

Roofline analysis, CUDA microbenchmarks, Nsight Compute counters, SAIT PIMSimulator, and problem-size sweeps integrated into one traceable workflow.

9	v6	31	2.36x
CUDA workloads	cache/stall-aware model	automated tests	modeled policy speedup

Executive Summary

This project goes beyond classifying a GPU kernel as memory-bound or compute-bound. It asks whether the kernel is a reasonable PIM/NMP exploration candidate and records the evidence behind that decision. CUDA events provide measured GPU runtime, the Roofline model establishes a first-order bound, and Nsight Compute counters refine the diagnosis using DRAM, cache, scheduler, and dependency-stall behavior.

Final outcome. Seven workloads remain PIM/NMP exploration candidates: vector addition, SAXPY, random gather, reduction, scan, matrix transpose, and GEMV. The high-reuse tiled GEMM and optimized cuBLAS SGEMM remain on the GPU. The v6 policy separates all seven positive labels from two negative controls on the calibration set. This is calibration alignment, not held-out generalization accuracy.

Key results

Item	Result	Interpretation
Measured GPU-only runtime	10.963 ms	Sum of nine CUDA-event measurements
Final v6 policy estimate	4.638 ms / 2.36x	Four simulator-backed and three analytical offloads
Simulator coverage	4/9 (44.4%)	vector_add, saxpy, reduction, and gemv
Calibration controls	3/3 pass	GEMV positive; tiled and cuBLAS GEMM negative
Size stability	Three scales	The 7 PIM/NMP to 2 GPU split remains stable

Report map

1. Problem and background | 2. System architecture | 3. Method | 4. Model evolution | 5. Experimental setup | 6. Roofline results | 7. NCU and v6 results | 8. Simulator normalization | 9. Size sweep | 10. Validation and limitations | 11. Reproduction | 12. Conclusion

1. Problem Definition and Background

1.1 Why GPU bottleneck classification matters

GPUs hide memory latency by scheduling many warps and provide high arithmetic throughput through many parallel execution units. A slow kernel, however, is not necessarily slow for the same reason as another kernel. Low arithmetic intensity can make DRAM bandwidth the limit. High arithmetic intensity can make compute throughput the limit. A kernel that uses neither resource effectively may instead be constrained by irregular access, cache misses, dependencies, divergence, low occupancy, or synchronization.

1.2 What PIM/NMP attempts to change

Processing-in-Memory and Near-Memory Processing reduce data movement by performing selected operations close to memory. The strongest candidates therefore tend to have low arithmetic intensity, large memory traffic, limited cache reuse, or high memory-latency stalls. Dense matrix multiplication is often a poor candidate because GPUs exploit its reuse and compute density effectively.

Signals favoring the GPU

- High arithmetic intensity
- High L2 or shared-memory reuse
- Compute-heavy operations
- Optimized library support

Signals favoring PIM/NMP exploration

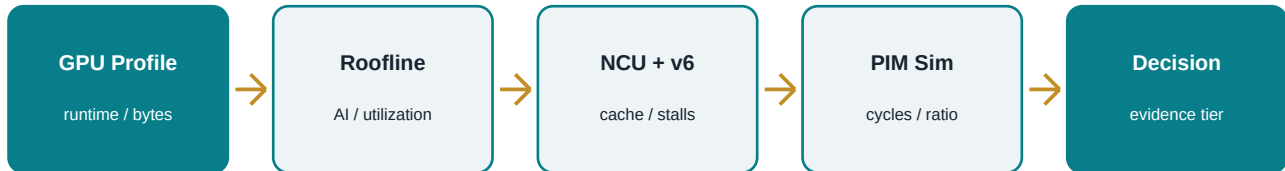
- Low arithmetic intensity
- DRAM pressure or memory stalls
- Streaming or irregular access
- Low reuse and high data movement

1.3 Project questions

Can measurable features separate GPU bottleneck classes? Can a cache-, stall-, reuse-, and transfer-aware model improve on low-AI rules? Can GPU measurements and PIM simulation be combined without mixing unrelated clocks? Are decisions stable across multiple problem sizes?

2. System Architecture

The repository separates data collection, feature extraction, model comparison, simulator attachment, policy evaluation, and reporting. Existing CSV files can be analyzed on a machine without CUDA, while the GPU server is responsible only for generating fresh measurements.



Measured, simulated, and estimated values remain traceable

Figure 1. Data flow from GPU measurement to the final evidence-aware decision

2.1 Input layer

Input	Role	Current basis
gpu_profile.csv	Runtime, FLOPs, theoretical DRAM bytes	RTX 2080 Ti CUDA events
ncu_metrics.csv	DRAM, cache, scheduler, and stall counters	Nsight Compute 2020.1.1
gpu_benchmark_metadata.csv	Complexity, reuse, transfer, and control roles	Calibration metadata
sait_pim_simulation.csv	PIM-on/PIM-off cycles and speedup	SAIT PIMSimulator logs

2.2 Output layer

Each full run produces a Roofline figure, model-comparison figure, end-to-end policy figure, and Markdown report. The size sweep stores three raw profiles, three detailed reports, a combined CSV and Markdown summary, and a trend figure. Intermediate evidence is retained so the final result can be traced to its source.

3. Analysis Method

3.1 Roofline first-stage classification

Arithmetic intensity is the amount of floating-point work performed per byte transferred from DRAM. The RTX 2080 Ti configuration uses 13.45 TFLOP/s peak FP32 throughput and 616 GB/s peak memory bandwidth, producing a ridge point of 21.8344 FLOP/Byte.

$$AI = \text{FLOPs} / \text{DRAM bytes}$$

$$\text{Attainable performance} = \min(\text{peak FLOP/s}, \text{peak bandwidth} \times AI)$$

$$\text{Roofline utilization} = \text{achieved performance} / \text{attainable performance}$$

3.2 Cache/stall-aware feature_cost_v6

v6 extends the reuse-aware v4 model and the basic NCU-aware v5 model. It optionally uses cache hit rates, load/store efficiency, warp and branch efficiency, long and short scoreboard stalls, barrier stalls, eligible warp supply, register pressure, and transaction proxies. The available optional-counter ratio controls how far v6 may move from the v5 estimate.

Signal group	PIM opportunity	GPU-resident signal
Cache	Low L1/L2 hit rate	High locality or reuse
Memory	High DRAM pressure or dependency stalls	Efficient coalescing and locality
Scheduler	Low eligible-warp supply	GPU can hide latency
Control	Low divergence and barrier risk	Complex control or synchronization
Metadata	Low complexity and high partitionability	Compute, transfer, or reuse risk

3.3 Common-cost policy evaluation

AI-only, traffic-only, heuristic, analytical, and feature-cost models choose different workloads to offload. Their total runtime is evaluated with the same current v6 PIM timing table. This isolates decision quality from differences between timing formulas.

4. Implementation and Model Evolution

The project evolved from a basic Roofline analyzer into an evidence-aware GPU/PIM research pipeline. Each model version addressed a concrete false positive, missing measurement, or comparability problem found in the preceding version.

Stage	Implementation	Problem addressed
v1-v2	Roofline, heuristic score, and PrIM-inspired proxy	Connect bottlenecks to literature categories
v3	GPU/PIM memory, compute, transfer, and sync cost	Reduce low-AI false positives
v4	Data-reuse gate and cuBLAS SGEMM control	Avoid misclassifying reusable GEMM
v5	Nsight Compute DRAM/SM/cache pressure	Calibrate metadata proxies with counters
v6	Cache hit, scheduler, stalls, and coverage	Separate bandwidth, locality, and latency causes
Final	Simulator ratio normalization and evidence tiers	Stop directly adding unrelated clock domains

Key engineering decisions

- Store CUDA-event runtime separately from theoretical FLOP and byte counts.
- Integrate external PIM tools through a small adapter CSV instead of vendoring a simulator.
- Preserve missing NCU values as NA for compatibility with older profiler versions.
- Parameterize benchmark sizes and iteration counts for repeatable experiments.
- Protect parsers, models, simulator normalization, and policy contracts with 31 automated tests.

Code organization

Area	Primary files
Analysis	src/roofline.py, classifier.py, features.py
Models	src/cost_model_v3.py through cost_model_v6.py
Integration	src/model_comparison.py, simulator.py, end_to_end.py
Measurement	benchmarks/*.cu and scripts/profile_*.sh
Reporting	src/plot.py, src/report.py, and main.py

5. Experimental Setup and Workloads

GPU server

- Ubuntu 20.04
- GeForce RTX 2080 Ti
- Driver 535.230.02
- CUDA toolkit 11.0
- Host compiler g++-9

Profiling stack

- CUDA event runtime
- nvprof raw logs
- Nsight Compute 2020.1.1
- SAIT PIMSimulator
- Python, pandas, and matplotlib

5.1 Benchmark suite

Workload	Purpose	Final role
vector_add / saxpy	Streaming bandwidth	PIM/NMP candidates
random_gather	Irregular memory latency	PIM/NMP candidate
reduction / scan	Collective memory primitives	Candidates with synchronization risk
matrix_transpose	Memory-dominated layout conversion	PIM/NMP candidate
gemv	Low-reuse matrix-vector multiply	PIM-positive control
matrix_mul_tiled	Reuse-aware custom GEMM	GPU negative control
cublas_sgemm	Optimized dense compute	GPU negative control

5.2 Problem-size sweep

Vector inputs use 1,048,576, 4,194,304, and 16,777,216 elements. Matrix inputs use dimensions 512, 1024, and 1536. The sweep compares runtime, achieved bandwidth, bottleneck class, offload decision, and analytical speedup trend.

5.3 Interpretation rule

Measured refers only to GPU runtime and profiler counters. Simulated refers to PIMSimulator cycles and ratios. Estimated refers to feature-cost and end-to-end policy outputs. The report maintains this vocabulary to avoid presenting model output as physical PIM measurement.

6. Roofline and GPU Measurement Results

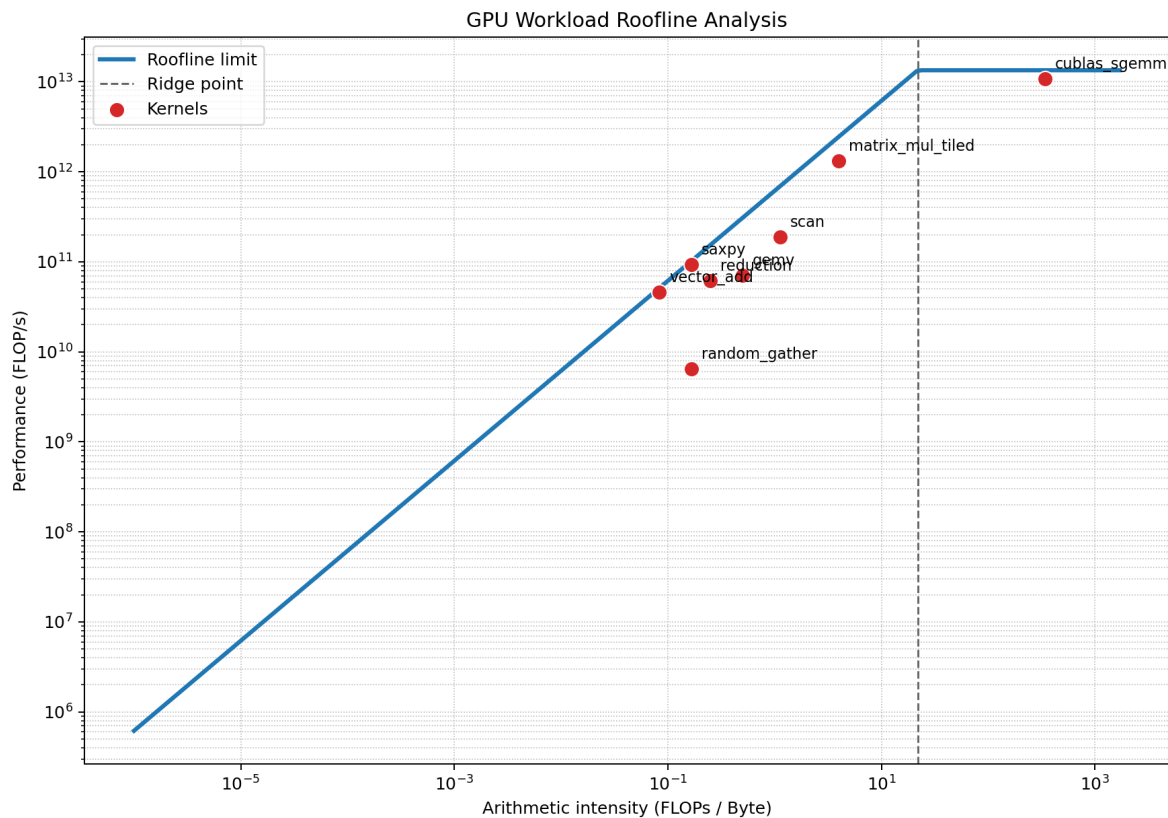


Figure 2. Roofline positions of the nine workloads measured on the RTX 2080 Ti

Group	Representative result	Diagnosis
Streaming	vector_add/saxpy near 555 GB/s and 90% Roofline	DRAM bandwidth saturation
Irregular	random_gather 38.8 GB/s and 6.3% Roofline	Latency/locality rather than bandwidth
Collective	reduction 40%; scan 27% Roofline	Mixed memory and synchronization effects
Dense	cuBLAS SGEMM AI 341.3 and 10.85 TFLOP/s	Strong compute-bound control
Reuse	tiled GEMM AI 3.97 and L2 hit 98.34%	Low AI alone must not imply PIM

Roofline alone places tiled GEMM in the memory region, yet its high L2 reuse makes it a poor PIM candidate. Random gather does not saturate DRAM but suffers extreme long-scoreboard stalls, making memory-latency reduction relevant. These two cases motivate an NCU-aware model.

7. Nsight Compute and v6 Results

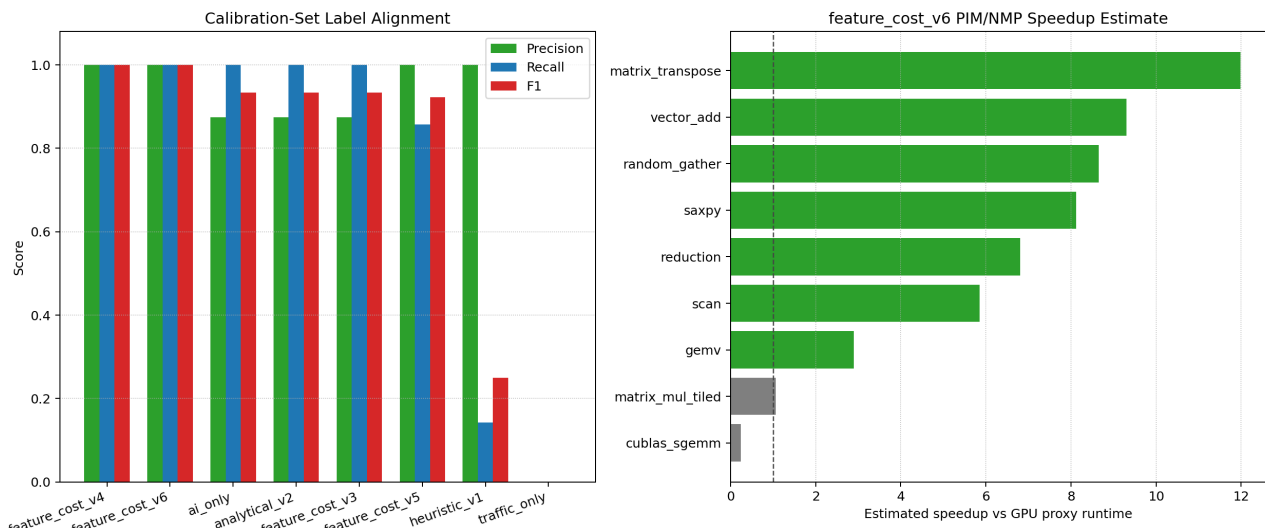


Figure 3. Calibration-set alignment and the latest v6 analytical speedup estimates

Kernel	Key NCU evidence	v6 interpretation
vector_add	SOL DRAM 85.41%; long scoreboard 91.20%	Streaming PIM opportunity
random_gather	L2 hit 9.99%; long scoreboard 98.30%	Irregular latency opportunity
gemv	L1 hit 87.78%; long scoreboard 67.50%	Some locality, but simulator-positive
matrix_mul_tiled	L2 hit 98.34%	Strong GPU locality advantage
cublas_sgemm	L2 hit 87.47%; compute-bound	Remain on GPU

7.1 Final offload decisions

PIM/NMP exploration candidates	Remain on GPU
vector_add, saxpy, random_gather	matrix_mul_tiled
reduction, scan, matrix_transpose, gemv	cublas_sgemm

The installed NCU version provides four of twelve optional v6 counters per kernel, or 33% coverage. v6 uses the cache and stall evidence but limits its correction from v5 by that coverage ratio. Missing values are not treated as measured zeros.

8. PIM Simulator Integration and Time Normalization

8.1 The comparability problem

The original integration converted SAIT PIMSimulator cycles using a temporary 1 ns/cycle value and directly combined that time with RTX 2080 Ti milliseconds. The simulator baseline and GPU are not the same architecture or clock domain. The conversion is useful for traceability but not for direct cross-platform addition.

```
raw_sim_time_ms = PIM cycles x cycle_time_ns / 10^6
scaled_PIM_time_ms = measured_GPU_time_ms / simulator_speedup
```

The final implementation preserves raw cycles and raw converted time as evidence. End-to-end totals use the simulator's internal PIM-on versus PIM-off speedup ratio applied to measured GPU runtime. Unmapped selected kernels use the v6 analytical estimate, and the report counts those fallbacks explicitly.

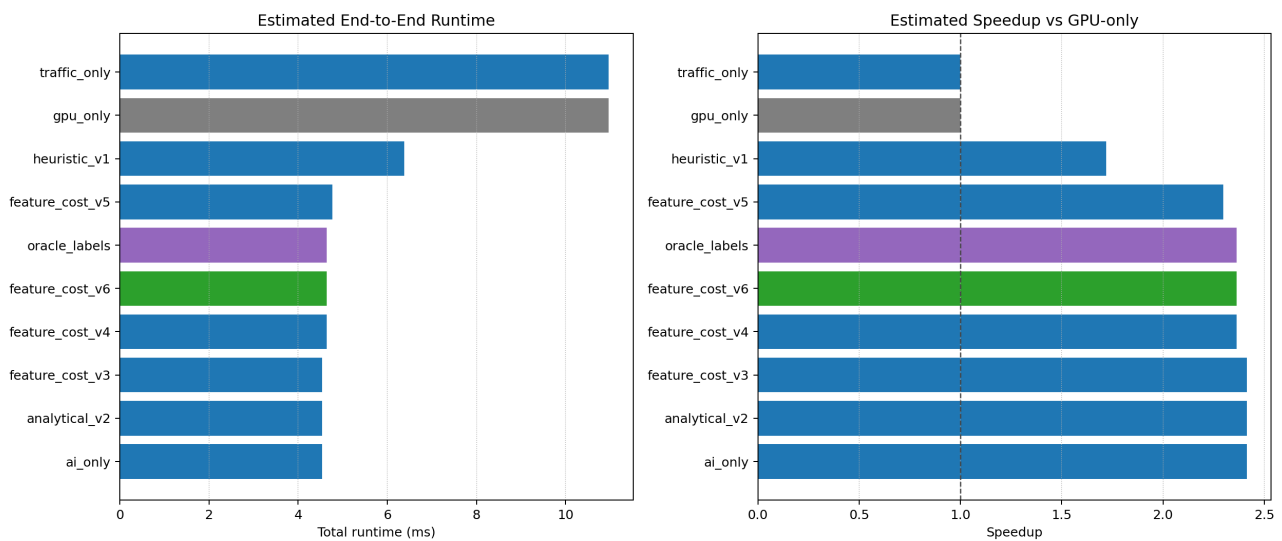


Figure 4. Offload policies evaluated with a common PIM timing table

Metric	Final value	Evidence
GPU-only	10.963 ms	Measured CUDA-event sum
feature_cost_v6 policy	4.638 ms	Four simulator-scaled and three analytical offloads
Modeled speedup	2.36x	Not measured PIM silicon speedup
Simulator coverage	44.4%	Four of nine mapped workloads

9. Problem-Size Sweep

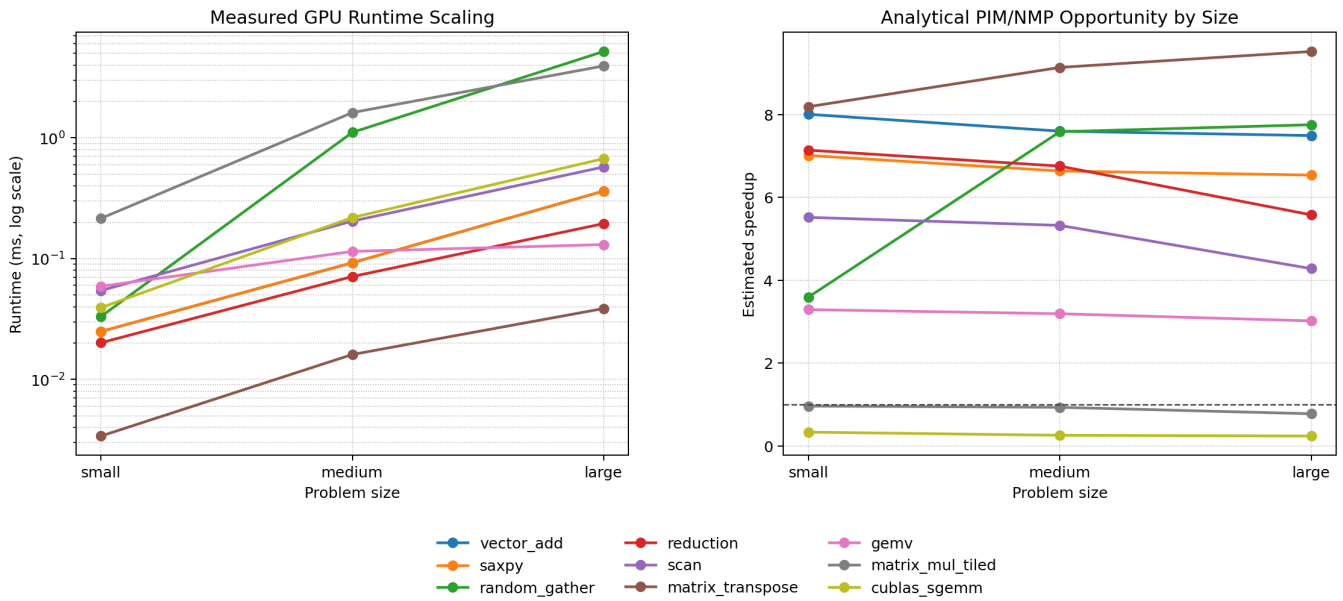


Figure 5. Measured GPU runtime and analytical PIM opportunity at three input sizes

Runtime increases normally with input size, with random gather showing the steepest growth. Streaming kernels preserve analytical opportunity at large sizes. Matrix transpose remains favorable, while tiled GEMM and cuBLAS SGEMM remain below 1x and stay on the GPU.

Workload	Small	Medium	Large
vector_add	PIM/NMP	PIM/NMP	PIM/NMP
random_gather	PIM/NMP	PIM/NMP	PIM/NMP
reduction	PIM/NMP	PIM/NMP	PIM/NMP
scan	PIM/NMP	PIM/NMP	PIM/NMP
gemv	PIM/NMP	PIM/NMP	PIM/NMP
matrix_mul_tiled	GPU	GPU	GPU
cublas_sgemm	GPU	GPU	GPU

The 7 PIM/NMP to 2 GPU decision split remains unchanged at all three sizes. This shows internal size stability within the current synthetic suite. It does not replace independent validation on unseen application workloads.

10. Validation, Evidence, and Limitations

10.1 Validation scope

Validation	Result	Residual risk
Automated tests	31 passed	GPU and NCU execution still requires the server
Calibration labels	v4/v6 F1 = 1.00	Not held-out accuracy
Control workloads	3/3 pass	Small control set
Size sweep	Three scales stable	Limited application diversity
PIM simulator	4/9 mapped	Mapping fidelity and hardware differences

10.2 Evidence tiers

Tier	Evidence	Current workloads
A	Measured GPU + NCU + mapped PIM simulator	vector_add, saxpy, reduction, gemv
B	Measured GPU + NCU + analytical PIM	Remaining five final workloads
C	Measured GPU + analytical PIM	Size-sweep runs without NCU

10.3 Limitations that must remain explicit

- No runtime was measured on physical PIM silicon.
- Nine microbenchmark-style workloads cannot establish generalization.
- FLOP and byte counts are theoretical implementation-level counts.
- Nsight Compute 2020.1.1 provides partial optional-counter coverage.
- Analytical bandwidth, throughput, transfer, and synchronization parameters require calibration.
- The end-to-end model does not fully include placement, orchestration, programming, or deployment overhead.

10.4 Highest-value next research step

The next priority should not be a feature-heavy v7. Freeze the current thresholds, evaluate unseen application-level workloads, repeat on another GPU architecture, and add measurements from UPMEM or another physical PIM platform before making hardware-level performance claims.

11. Reproduction

11.1 Run on the GPU server

```
cd ~/GPU-Workload-Bottleneck-Analyzer
git pull
source .venv/bin/activate
pip install -r requirements.txt
python3 -m pytest
bash scripts/build_benchmarks.sh
bash scripts/profile_nvprof.sh
NCU_EXTRA_ARGS="--section MemoryWorkloadAnalysis --section SchedulerStats --section WarpStateStats" \
bash scripts/profile_ncu.sh
python3 scripts/parse_ncu_reports.py --input-dir profiles/ncu --output profiles/ncu_metrics.csv
python3 scripts/parse_sait_pim_logs.py --log-dir ~/pim-tools/pim-results \
--output simulators/sait_pim_simulation.csv --cycle-time-ns 1.0
python3 main.py --input profiles/gpu_profile.csv \
--paper-baseline paper_baselines/gpu_benchmark_metadata.csv \
--pim-simulation simulators/sait_pim_simulation.csv \
--ncu-metrics profiles/ncu_metrics.csv \
--output-dir outputs/gpu_profile_with_sait_pim_ncu \
--hardware-name "RTX 2080 Ti" --peak-flops 13450000000000 \
--peak-memory-bandwidth 616000000000
bash scripts/profile_size_sweep.sh
```

11.2 Fetch results to the local Mac

```
cd /Users/kingjung/Desktop/gpu-bottleneck-analyzer
git pull
bash scripts/fetch_server_results.sh gpu_profile_with_sait_pim_ncu
open outputs/gpu_profile_with_sait_pim_ncu/reports/analysis_report.md
open outputs/size_sweep/size_sweep_summary.md
open outputs/size_sweep/size_sweep.png
open output/pdf/gpu_pim_bottleneck_analyzer_portfolio_report_en.pdf
```

11.3 Success criteria

A complete run contains nine rows in `gpu_profile.csv`, nine kernels in `ncu_metrics.csv`, four simulator mappings, three final figures, a Markdown report, three size-sweep profiles, and the combined size-sweep summary and figure. The test command must report 31 passed.

12. Conclusion

The GPU Workload Bottleneck Analyzer connects GPU performance diagnosis, memory-system counter interpretation, and PIM/NMP candidate selection in one reproducible tool. Its central contribution is the progression from low-AI rules to a reuse-aware model, an NCU cache/stall-aware v6 policy, and a clock-domain-safe simulator integration.

Project quality is not defined by the largest speedup number. It is defined by whether the result can be reproduced, traced to evidence, and explained with its limitations. The repository is complete as a portfolio-grade research prototype. Its next stage is broader external validation, not additional unvalidated model complexity.

References

- [1] S. Williams, A. Waterman, and D. Patterson, 'Roofline: An Insightful Visual Performance Model for Multicore Architectures,' CACM, 2009.
- [2] NVIDIA, Nsight Compute Kernel Profiling Guide: Roofline, Memory Workload Analysis, and Scheduler Statistics.
- [3] J. Gomez-Luna et al., 'Benchmarking a New Paradigm: An Experimental Analysis of a Real Processing-in-Memory Architecture,' IEEE Access, 2022.
- [4] Samsung Advanced Institute of Technology, SAITPublic/PIMSimulator.
- [5] NVIDIA, CUDA C++ Programming Guide and cuBLAS documentation.

Repository artifact: GPU-Workload-Bottleneck-Analyzer

Final implementation includes source, tests, raw and parsed profiles, simulator adapters, generated reports, and bilingual portfolio documents.